Kubernetes Probes and Startup Failures

Day 01 | KUBERNETES / EKS AKS GKE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

A probe is a control decision, not just an HTTP request. Startup asks whether initialization finished; readiness controls ordinary traffic eligibility; liveness asks whether a restart may recover the process. Giving these checks different endpoints makes their intent explicit.

What Goes Wrong Repeated liveness failures can restart the container before this 60-second startup completes.

Check The Cause Do not assume every restart is a probe failure. Check events, exit reasons and previous logs.

Before you start

A test Kubernetes namespace; access to Pod logs and events; an app with startup, readiness and liveness endpoints; measured startup timings.

Diagnostic checks

```
kubectl describe pod <pod>
```

```
kubectl logs <pod> -c <app> \
  --previous
```

Look For: Liveness probe failed and a container restart. Check the probe path, port and app startup logs.

Rule Out: A crashing process, OOM kill or incorrect configuration needs its own fix. More startup time will not repair those.

Apply the cause specific fix

Give Startup Its Own Check

```
# Add under the app container
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  periodSeconds: 5
  failureThreshold: 30
  timeoutSeconds: 2
```

Startup Window: 30 failures x 5 seconds gives a nominal 150-second budget. Choose values from measured startup time.

Why It Helps: Liveness and readiness probes wait until the startup probe succeeds. After startup, their normal checks begin.

Ready For Traffic Or Needs A Restart

```
# Same app container
readinessProbe:
  httpGet: {path: /ready, port: 8080}
  periodSeconds: 5
  timeoutSeconds: 2
livenessProbe:
  httpGet: {path: /live, port: 8080}
  periodSeconds: 10
  failureThreshold: 3
  timeoutSeconds: 2
```

Readiness: Failure marks the Pod unready for normal Service routing. It does not itself restart the container.

Liveness: Failure past the threshold triggers a restart. A shared database outage should not automatically cause every app to restart.

Validate the result

```
kubectl rollout status \
  deployment/api

kubectl get pods -w

kubectl describe pod <pod>
```

Expected Result: Initialization finishes, readiness becomes true, and startup-related restarts stop.

Test In A Lab: Delay startup and confirm traffic waits. Fail readiness and confirm no restart. Test liveness recovery separately.

Practice exercise

Make the test app take 60 seconds to initialize. Compare premature liveness restarts with the startup-protected rollout. Then force readiness failure and confirm the container does not restart.

Implementation notes

A startup probe suppresses liveness and readiness checks until it succeeds. The 150-second example is a nominal failure budget; exact timings include probe and termination behaviour. The app must implement the selected HTTP endpoints. Readiness failures do not themselves restart a container. Liveness should not blindly depend on a shared database. A long startup allowance will not repair a crash, OOM condition or invalid configuration.

Interview question

Why can one shared dependency check be dangerous as a liveness probe? Explain how restarting every caller can amplify the original dependency outage.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1

Official documentation 2